1. What is React? & Why it is it called a library instead of a framework? Explain component based architecture, declarative UI, and the problems React solves.

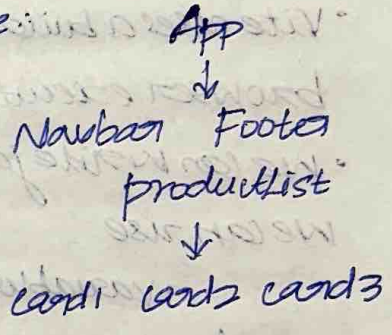
- React is a javascript library used to build user interfaces.
- React makes creating & updating the elements easier by allowing you to divide the UI into reusable components.

Framework provides a more complete structure and controls more of application's architecture. Whereas library is a collection of tools that you used when needed.

React primarily handles the UI layers. So, its a library.

A component is a reusable piece of UI. In a webpage we can create seperate components like navbar, footer, productlist. We can update and reuse it whenever needed.

Benifits of component based architecture:



→ Easy maintainance.
→ Organized code.
→ Easier to debug.
→ Independent UI sections.

Declarative UI:

- Declarative means describing what the UI should look like, rather than manually explaining every step to create it.

problems React solves:

→ Repeated code - reusable components.
→ Manual DOM manipulation - React manages UI updates.
→ Difficult UI maintainance - components organize code
→ Data synchronization - state & props help keep UI consistent with data.
→ complex interactions - React provides a structured way to handle them.

2. What is JSX, and how does the browser execute code written using JSX? Explain JSX transformation, javascript expressions inside JSX & how JSX differs from HTML.

- JSX stands for Javascript XML. It is a syntax extension for javascript that allows you to write HTML-like markup inside javascript.
- Browsers cannot directly understand JSX. JSX must first be transformed into regular javascript.

jsx code
↓
jsx transformation.
↓
javascript
↓
React renders UI.

- Vite uses a build toolchain to transform JSX before the browser executes the resulting javascript.
- We can write javascript expressions inside curly braces.

We can use:

- variables, math expressions, ternary operators, function calls, array methods.

- JSX has className, htmlFor, onClick={handleClick},
- javascript expressions use {}.
- components can be used as tags.

HTML:

- HTML has class, for, onclick functions.
- HTML does not use JSX expression syntax.
- HTML uses built in elements.

3. What is the difference between the Real DOM & Virtual DOM? Explain rendering, reconciliation and the how React decides which DOM elements to update.

DOM is stands for Document Object Model.
The Browser converts an HTML documents into a tree like structure called DOM.

- Virtual DOM is a memory representation of UI used by react. React uses this representation to calculate what changes are needed before updating the actual browser DOM.

Real DOM	Virtual DOM.
Actual browser document structure.	React's UI representation.
Managed by Browser.	Managed by React.
Represents actual webpage	Describes expected UI
DOM changes affect the page	used to calculate required changes.

- Rendering means React calls a component function to calculate what the UI should look like
- Reconciliation is the process of comparing the previous UI representation with the new one to determine what needs to be updated.

How React updates DOM elements?

State/props change
↓
Component Renders Again
↓
New UI representation
↓
Reconciliation
↓
React identifies changes
↓
Required DOM updates.

4. What causes React component to render & re-render? Explain the roles of state, props, parent rendering and why re-rendering does not always mean the DOM changes

causes of component re-render:

- State change: When a state setter updates the state.
- Props change: When a parent provides different props.
- Parent re-renders: child components normally renders again when their parent renders.
- Context changes: When a context value changes for a consuming component.

When the button is clicked.

↓

state changes

↓

React schedules a render

↓

component executes again

↓

React calculates the new UI.

↓

Necessary DOM updates are committed.

- No, re-rendering does not always update the DOM.
- Re-rendering means React recalculates the UI

If the resulting UI is the same, React may not need to make any DOM changes

5. What are props, and state, and how are they different? Explain one way data flow, immutability, lifting state up, and why directly modifying state causes problem.

- props are used to pass data from a parent component to a child component.
- State is data managed by a component that React remembers between renders

props	state
passed from parent	Managed by component
Read only to receiving component	updated using setter.
used to pass data.	used to manage changing data
can trigger re-renders.	can trigger re-renders.

- React follows one way data flows. It means data flows from parent to child through props.
- Immutability means not directly modifying existing state objects or arrays.
- Helps react detect state changes.
- Lifting state up means moving state to the nearest common parent when multiple child components need the same data.



Both receives same data through props.

6. How does useState() work? Explain state snapshots, batching functional updates & why reading state immediately after calling its setter can return the previous value

- useState() is a React hook used to add & manage state in functional components.
- State behaves like a snapshot for each render.
- Calling a setter does not immediately change the state variable in the currently executing function.
- Batching means react groups multiple state updates to reduce unnecessary renders.

- Functional updates calculate the next state using the previous state.

Advantages:

- correctly handles consecutive state updates.
- useful when new state depends on previous state.
- works reliably with batched updates.

7. How does useEffect() work, and when should it be used? Explain dependency arrays, cleanup functions, stale closures, and the additional effect setup and cleanup cycle in development StrictMode

- useEffect() is a React Hook used to synchronize a component with external systems.

uses:

- Api fetching.
- Timers.
- Event listeners.
- Browser API synchronization.

Dependency array:

No dependency - After every committed render.

[] - after initial mount.

[count] - After mount and when count changes.

- A cleanup function stops or removes resources created by an effect.

Uses:

- Clearing timers.
- Removing event listeners.
- Cancelling requests.

- A stale closure happens when an effect uses an outdated value captured from an earlier render.

→ Includes required reactive values in dependencies.
→ Use functional state updates where appropriate.

React strict mode:

- Strict mode is development tool used to identify potential problems.

8. What are controlled and uncontrolled components in React? Explain form handling, validation, the role of useRef(), and when each approach is appropriate.

- A controlled component is a form element whose value is controlled by React state.

Advantages:

- Easy validation.
- Live input tracking
- Easy error handling.
- conditional form handling.

- An uncontrolled component stores its value in the DOM rather than react state. React accesses the value using useRef().

Controlled	Uncontrolled
Uses usestate	often uses useRef.
uses value	uses defaultValue
uses onchange	Reads value & DOM manages value
Easy validation	Suitable for simpler forms.

Role of useRef().

useRef() creates a reference that persists between renders.

Uses:

- Accessing DOM elements.
- Focusing input fields
- Reading uncontrolled form values
- Storing values without triggering re-renders.

9. How does React render lists & conditional content? Explain map(), stable keys, problems with array-index keys, and how strings, numbers, booleans, null & undefined behave in JSX.

- List rendering means displaying multiple elements using an array. React commonly uses JavaScript's map().
- Keys are special identifies used for React to identify list elements between renders.
  - → Helps reconciliation.
  - → Helps React identify items.
- Array indexes can change when items are added, deleted or reordered. This can cause React to associate the wrong component state with a list item.

Conditional Rendering:

- → displaying UI based on condition.
  - using ternary operator.
  - using logical operators.

How javascript values behave in JSX

Value	JSX behaviour
String	Display text
Number	Display Number
true	Renders nothing
false	Renders nothing
null	Renders nothing
undefined	Renders nothing.

10. How does client-side routing work in React Routes? Explain BrowserRouter, Routes, Route, Link, NavLink, nested routes & outlet, including how navigations differ from a full page reload.

- React routes is a routing library used to navigate between different pages or views in a React application without a full page reload.

component	purpose
BrowserRouter	Enables browser-based routing
Routes	contains route definitions.
Route	Defines URL & components.
Link	Navigates between pages.
NavLink	Navigates & identifies active links.
Outlet	Displays matching nested child routes.

BrowserRouter.

Provides routing functionality using the browser history API.

Routes & Route:

Routes groups route definitions.
Routes connects a URL path with a component.

Link & NavLink

Link	NavLink
Used for navigation	used for navigation
No automatic active styling	provides active state
Suitable for normal links	Suitable for navigation menus.

- Nested routes are routes defined inside another route
- Outlet is a placeholder that displays the matching child route inside the parent component.

11. How do useNavigate(), useParams(), useLocation(), and useSearchParams() differ? Explain browser history, replace, true, dynamic path parameters, query parameters, and passing data through navigation state.

useNavigate():
Used to navigate programmatically from one route to another

useParams():
Used to access dynamic parameters from the URL

useLocation():
Used to access information about the current location

useSearchParams():
Used to read & update query parameters in a URL

React Router Hooks

Hook	Main purpose
useNavigate	Navigate between routes
useParams	Read dynamic path parameters

path parameters vs query parameters

path parameters	Query parameters
Part of URL path	Part of URL search string
identify a specific resource	Usually provide filters or options
/products/101	/products?category=mobile
Access using useParams	Access using useSearchParams